

The Visual Sentinel: Project Documentation

Executive Summary

The Visual Sentinel is a comprehensive automated testing project that leverages Playwright and Percy to implement **Visual Regression Testing (VRT)**. Standard functional tests often pass even when the User Interface (UI) is visually broken (e.g., misaligned elements, broken CSS, layout shifts). This project establishes a 'safety net' to automatically detect these "visual bugs" by comparing snapshots of the UI against an approved baseline, ensuring consistency and preserving brand integrity across every deployment.

Project Goal and Objective

Category	Description
Primary Goal	To implement a robust, automated Visual Regression Testing pipeline for web applications.
Key Objective 1	Successfully integrated Playwright for browser automation and Percy for visual snapshot comparison.
Key Objective 2	Establish a reliable VRT baseline for key application pages (e.g., login, dashboard).
Key Objective 3	Integrate the VRT process into a continuous integration (CI/CD) workflow using GitHub Actions.
Success Metric	The system automatically detects and reports intentional CSS/layout changes upon the second test run, and the CI/CD pipeline executes the tests successfully on push/pull request events.

Technology Stack

The project utilizes a modern, industry-standard stack for end-to-end testing and VRT.

Technology	Role
Playwright	Browser automation, test execution, page interaction (Go to URL, evaluate scripts).

Technology	Role
Percy.io	Visual comparison engine, snapshot storage, diff highlighting, and VRT workflow management.
Node.js/npm	Project environment and dependency management.
GitHub Actions	Continuous Integration (CI) service for automated execution on code changes.
JavaScript	Test scripting language.

Implementation Roadmap

This roadmap outlines the steps taken to build and automate the Visual Sentinel system.

Phase	Description	Status
Phase 1	Local Setup (Project Initialization, Playwright/Percy installation, baseline test creation).	Complete
Phase 2	Connecting the Brain (Percy account creation, token configuration, and running the initial baseline test).	Complete
Phase 3	Simulating a "Visual Bug" (Test modification to inject CSS change and running a comparison test).	Complete
Phase 4	Automating it (CI/CD integration via GitHub Actions, workflow file creation, and secret management).	In Progress
Phase 5	Expansion (Adding tests for multiple pages and different screen resolutions).	To Do

Detailed Implementation Guide (Reference)

Phase 1: Local Setup (The Engine)

The foundational environment is established using Node.js and key dependencies.

Step	Action	Command/Description
1.1	Initialize Project	<code>mkdir visual-sentinel && cd visual-sentinel && npm init -y</code>
1.2	Install Tools	<code>npm install -D @playwright/test @percy/cli @percy/playwright</code>
1.3	Install Browsers	<code>npx playwright install</code>
1.4	Create Test File	Create <code>visual.spec.js</code> with the baseline test code targeting <code>https://www.saucedemo.com/</code> .

Phase 2: Connecting the Brain (Percy)

Percy is configured as the VRT comparison service.

Step	Action	Key Outcome
2.1	Account Setup	Percy project created, and <code>PERCY_TOKEN</code> obtained.
2.2	Baseline Run	The first VRT run executed, establishing the visual Truth .

Baseline Execution Command (Linux/Mac):`export
PERCY_TOKEN="<paste_your_token_here>"`

`npx percy exec -- npx playwright test`

Phase 3: Simulating a "Visual Bug"

This phase demonstrates the VRT system's ability to catch injected visual defects.

Step	Action	Change Implemented
3.1	Modify Test	Added <code>page.evaluate()</code> script to modify the login button's <code>backgroundColor</code> and <code>marginLeft</code> .
3.2	Comparison Run	Executed the test again without changing the Percy token.
3.3	Verification	Confirmed "Changes Detected" alert in the Percy Dashboard, viewing the highlighted visual diff.

Phase 4: Automating it (CI/CD)

The VRT is integrated into the development workflow for continuous protection.

Step	Action	File/Configuration
4.1	Create Workflow	Created <code>.github/workflows/main.yml</code> .
4.2	Configure YAML	Added jobs for dependency installation, browser setup, and test execution using the <code>PERCY_TOKEN</code> secret.
4.3	Secure Token	Added the <code>PERCY_TOKEN</code> as a secret in the GitHub repository settings.

Future Enhancements (Phase 5)

Enhancement	Description	Priority
Cross-Browser/Viewport VRT	Extend snapshots to capture different resolutions (e.g., mobile, tablet, desktop) and browsers (e.g., Chromium, Firefox, WebKit).	High

Enhancement	Description	Priority
Visual Scope Targeting	Implement options to only snapshot specific components or regions of a page, rather than the whole viewport, for faster and more focused comparisons.	Medium
E2E Flow VRT	Integrate visual snapshots into end-to-end flows (e.g., <code>login</code> -> <code>dashboard</code> -> <code>take_snapshot</code>).	Medium
Documentation & Reporting	Set up a formal reporting structure for Percy builds and integrate status checks back into GitHub Pull Requests.	Low